

Product builder

Ejercicio 2

Objetivo

Entraste como desarrollador backend a **Bibliotk**, una plataforma de reseñas de libros. Perfecta semana para llegar: hace tres días un autor pagó una campaña de reseñas falsas, el promedio de su libro pasó de 2.1 a 4.9 estrellas en cuatro horas, y mientras eso ocurría la home tardó 30 segundos en cargar. El sistema de moderación baneó las cuentas al día siguiente, pero el promedio del libro sigue en 4.9.

Tu trabajo es rehacer el motor de calificación: que los promedios sean correctos, que sigan siendo correctos cuando alguien es baneado después de haber reseñado, y que todo esto aguante cuando la próxima campaña llegue — porque va a llegar.

Brief

Los usuarios escriben reseñas de libros. Una reseña tiene una calificación en estrellas y, opcionalmente, un texto. Cada libro muestra el promedio de sus calificaciones.

Los usuarios pueden ser baneados por el equipo de moderación, y casi siempre eso ocurre *después* de que ya escribieron sus reseñas. Un usuario baneado puede tener miles de reseñas repartidas por el catálogo.

La home lista 50 libros con su promedio. Los títulos populares superan las 500.000 reseñas.

Tareas

- Implementa el sistema en **Ruby on Rails**. Solo backend: no se necesita frontend. Cómo expones el sistema hacia afuera es decisión tuya, mientras un cliente pueda consumirlo.
- Reseñas:
 - Una reseña tiene una calificación entera de 1 a 5 estrellas y un contenido de texto opcional de máximo 1000 caracteres.
 - Un usuario puede dar como máximo una reseña por libro.
 - Un usuario puede editar el rating y el contenido de su reseña, y puede eliminarla.
- Cálculo del promedio:
 - El promedio se expresa con un decimal, con redondeo half-up (3.25 → 3.3).
 - Un libro con menos de 3 reseñas muestra "Reseñas Insuficientes" en lugar del promedio.
 - Las reseñas de usuarios baneados no cuentan para el promedio.
 - Leer el promedio de un libro no puede recorrer sus reseñas: listar los 50 libros de la home con su promedio debe resolverse con una cantidad de consultas independiente del número de libros listados y del número de reseñas que cada uno tenga.

- Baneos:
 - Se puede banear y desbanear usuarios.
 - Banear o desbanear debe quedar reflejado en los promedios de todos los libros que ese usuario reseñó.
- Concurrencia y consistencia:
 - Si 200 usuarios distintos reseñan el mismo libro simultáneamente, al terminar el promedio y el conteo de ese libro deben ser correctos. No basta con que "casi siempre" lo sean.
 - Registrar una reseña debe responder rápido.
- Tests:
 - Usa RSpec.
 - Todo comportamiento no trivial debe tener un test que falle sin tu implementación.
 - Como mínimo, cubre: el redondeo en sus bordes, el umbral de las 3 reseñas, el baneo retroactivo, el ciclo editar/eliminar, y el invariante de unicidad bajo concurrencia.

Entregables

1. El código, en un repositorio con instrucciones para levantarlo y correr los tests.
2. Un `DECISIONES.md` breve con:
 - Los requisitos de este enunciado que te resultaron ambiguos o contradictorios, y qué decidiste en cada caso.
 - Los trade-offs que tomaste y qué costo tiene cada uno.
 - Qué dejarías fuera si esto saliera a producción mañana, y qué harías distinto con una semana más.

Bonus

- Un seed que genere un libro con 500.000 reseñas, y una medición que demuestre que el listado de la home no se degrada con él en el catálogo.
- Qué pasa si el mismo autor vuelve a comprar reseñas: ¿cómo detectarías la anomalía?

Uso de IA

Puedes usar asistentes de IA sin restricción. En la entrevista de revisión te vamos a pedir que:

- Expliques cualquier línea del código que entregaste y qué pasa si se elimina.
- Cuentas qué sugerencias de la IA descartaste y por qué.

No evaluamos si usaste IA, sino si entiendes lo que entregaste.

Alcance

Esperamos entre 4 y 6 horas de trabajo. Si algo no alcanza a entrar, prefíerelo bien resuelto y documentado antes que completo y a medias: déjalo escrito en `DECISIONES.md`.

Criterios de evaluación

- **Compleitud:** ¿implementaste lo pedido?
- **Correctitud:** ¿el sistema se comporta bien en los casos bordes, no solo en el camino feliz?
- **Criterio:** ¿tomaste decisiones defendibles?

- **Mantenibilidad:** ¿el código es claro y modificable por otra persona?
- **Testing:** ¿los tests prueban comportamiento real o solo acompañan al código?

Éxito y buen código,

El equipo de ComunidadFeliz